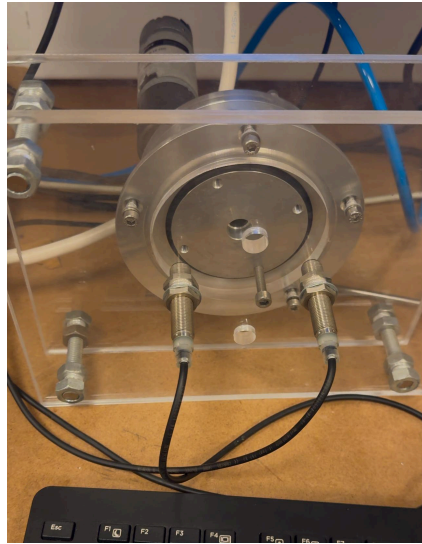


ME308 - Industrial Control

Lab #7 - Sequential Control System for Industrial Setup

Göktuğ Arda Gök
30975

Section - B



Faculty of Engineering and Natural Sciences
Mechatronics Engineering

1. Introduction

This report describes the design and implementation of a **sequential control system** for an industrial motor setup using a Beckhoff panel with touch control and hardware components. The system utilizes a series of inputs, outputs, and timers to coordinate motor actions based on sensor feedback. The system features:

- **Soft Start and Stop Buttons** to control the system operation,
- **Lock Button** to unlock the system before starting,
- **Proximity Sensors** (prox1 and prox2) to detect positions,
- **Signal Lamp** for visual feedback,
- **Relays** to control motor operation and direction.

The program operates in a sequence of states to achieve controlled motor movement, manual operation, and lamp signaling.

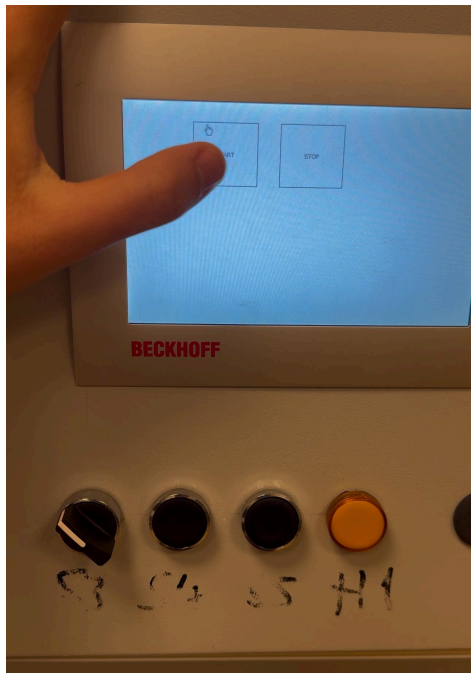


Figure 1: Button Setup

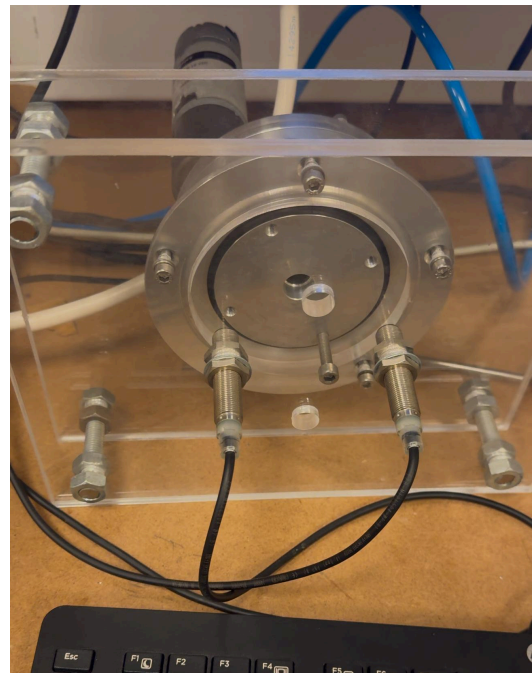


Figure 2: Motor and Sensor Setup

2. Process

The program is modularized into four main **Function Blocks (FBs)** to achieve separation of concerns:

1. **c001** - Coordinator: Handles system locking, unlocking, and transitions between states.
2. **m001** - Detect Bolts: Controls the motor behavior based on proximity sensor feedback.
3. **m002** - Manual Control: Allows manual motor operation with direction control and timing.
4. **m003** - Blink Lamp and Bolt Counting: Handles the blinking of the lamp and counting bolts based on prox2 triggers.

The state transitions and motor operations are controlled step-by-step based on input conditions and timers.

Visualization (V001)

A visualization panel was implemented to provide user interaction and feedback. The Beckhoff panel was used to display and control system states. The visualization includes:

- Start and Stop Buttons: Represented as touch buttons for starting and stopping the system.

Global Variables

The following inputs, outputs, and global variables are declared:

```
VAR_GLOBAL
// Inputs
btnStart : BOOL;    // Soft Start Button
btnStop : BOOL;     // Soft Stop Button
lockTrigger : BOOL; // Lock Button
btnCW : BOOL;       // Manual Clockwise Button
btnCCW : BOOL;      // Manual Counterclockwise Button
prox1 : BOOL;       // Proximity Sensor 1
prox2 : BOOL;       // Proximity Sensor 2

// Outputs
relayRun : BOOL;    // Motor Relay
relayDir : BOOL;    // Motor Direction Relay
lamp : BOOL;        // Signal Lamp

// Shared Variables
state_number : INT := 0; // Current State
boltCount : INT := 0;    // Bolt Counter
userN : INT := 3;        // Target Number of Bolts
END_VAR
```

Coordinator (c001)

The c001 function block coordinates system locking and state transitions:

1. **Unlocking**: When the lockTrigger is pressed, the system unlocks.
2. **Starting**: Pressing the btnStart starts the system and transitions to the first state.
3. **Stopping**: Pressing the btnStop stops the system immediately.

Code:

```
lockbutton(CLK := lockTrigger);  
IF lockbutton.Q THEN  
    relayRun := TRUE;  
    relayDir := FALSE;  
END_IF;
```

```
IF btnStart THEN  
    state_number := 1;  
END_IF;
```

```
IF btnStop THEN  
    state_number := 0;  
    relayRun := FALSE;  
    lamp := FALSE;  
END_IF;
```

```
IF state_number = 0 THEN  
    relayRun := FALSE;  
    lamp := FALSE;  
END_IF;
```

m001: Detect Bolts

The m001 function block controls the motor operation based on the proximity sensors:

1. The motor runs **clockwise** until prox1 detects a position.
2. After detecting prox1, the motor reverses direction to **counterclockwise**.
3. If prox2 is triggered, the state transitions to the next step.

Code:

```
IF prox1 THEN  
    relayRun := TRUE;
```

```

    relayDir := TRUE;
    prox1_detected := TRUE;
END_IF;

prox2Trigger(CLK := prox2);
IF prox1_detected AND prox2Trigger.Q THEN
    relayRun := FALSE;
    relayDir := FALSE;
    state_number := 2;
END_IF;

```

m002: Manual Motor Control

The m002 function block allows manual motor control with buttons btnCW (clockwise) and btnCCW (counterclockwise). It also incorporates timers to limit the motor runtime:

1. The motor runs in the selected direction based on button input.
2. A 10-second timer (m002_timer1) restricts manual operation.
3. After 10 seconds, the system waits for 5 seconds (m002_timer2) before transitioning to the next state.

Code:

```

manualControlTimer(IN := m002_timer1, PT := T#10S);
IF manualControlTimer.Q THEN
    IF btnCW THEN
        relayRun := TRUE;
        relayDir := FALSE;
    ELSIF btnCCW THEN
        relayRun := TRUE;
        relayDir := TRUE;
    ELSE
        relayRun := FALSE;
    END_IF;
END_IF;

m002_timer1_trigger(CLK := manualControlTimer.Q);
IF m002_timer1_trigger.Q THEN
    manualControlTimer2(IN := m002_timer2, PT := T#5S);
    relayRun := FALSE;
END_IF;

m002_timer2_trigger(CLK := manualControlTimer2.Q);

```

```
IF m002_timer2_trigger.Q THEN
    state_number := 3;
END_IF;
```

m003: Blink Lamp and Bolt Counting

The m003 function block manages the blinking lamp for visual feedback and counts bolts detected by prox2:

1. The lamp blinks every **500 ms** using a timer.
2. Every time prox2 triggers, the bolt count increments.
3. If the boltCount reaches the target (userN), the system transitions back to the first state.

Code:

```
prox2Trigger(CLK := prox2);
IF prox2Trigger.Q THEN
    boltCount := boltCount + 1;
END_IF;
```

```
IF boltCount >= userN THEN
    relayRun := FALSE;
    lamp := FALSE;
    state_number := 1;
    boltCount := 0;
END_IF;
```

```
blinkTimer(IN := TRUE, PT := T#500MS);
IF blinkTimer.Q THEN
    lamp := NOT lamp;
    blinkTimer(IN := FALSE);
END_IF;
```

3. Results

The program was executed successfully, and the following behavior was observed:

1. The system starts after pressing the **lock button** and **soft start button**. (figure 3) (figure 4)

2. The motor operates **clockwise** until prox1 detects a position (figure 5), and then reverses direction **counterclockwise** (figure 6), till prox2 detects a N-TRIG position (figure 7).
3. Manual motor control operates for **10 seconds** with buttons btnCW and btnCCW, followed by a **5-second wait** period. (figure 8)
4. The lamp blinks every **500 ms** during bolt counting, and the system counts bolts based on prox2 detection. (figure 9)

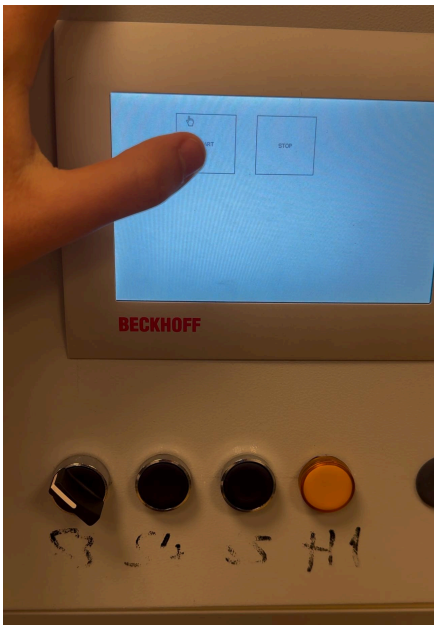


Figure 3: Clicking Soft Button Start



Figure 4: Lock Switch Opened

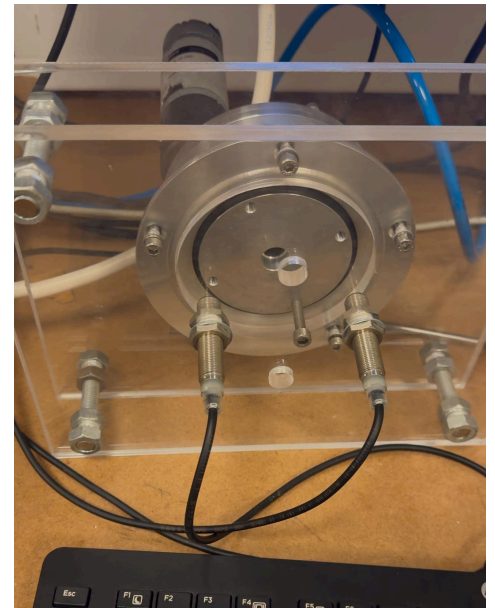


Figure 5: Motor Starts Turning Clockwise



Figure 6: Left Proximity Switch Active, Motor Starts Turning CCW

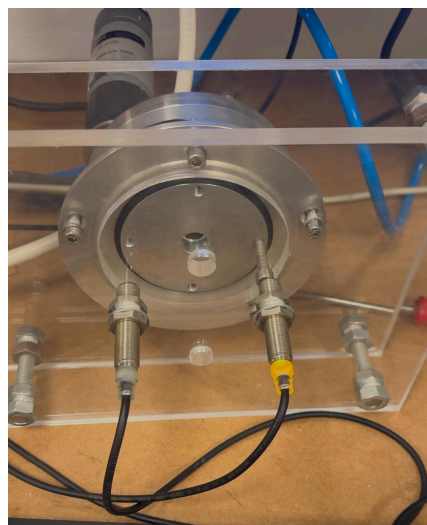


Figure 7: Right Proximity Switch Active, M002 starts, Motor Controlled Manually



Figure 8: Buttons to Control Motor, CW or CCW



Figure 9: H1 Starts Blinking

4. Discussion

The sequential control system operates efficiently using:

- **State Management:** States (state_number) ensure a clear progression through m001, m002, and m003. This enables modularity and maintainability.
- **Timers:** Timers are used for controlled delays, motor runtime limitations, and lamp blinking. The use of timers ensures reliable operation, prevents overuse of the motor, and provides visual feedback through precise timing.
- **Proximity Sensors:** The two proximity sensors (prox1 and prox2) play a critical role in detecting the motor's position and counting bolts. Their implementation ensures accuracy in system operation and state transitions.

The modular structure of the program (with function blocks c001, m001, m002, and m003) improves:

Additionally, the integration of a lock button adds safety to the system by preventing accidental starts. The soft stop button ensures the system can be safely halted at any time.

The system successfully met the project requirements and provided:

- Controlled motor operation in both automatic and manual modes.
- Clear visual feedback via the blinking lamp.

Overall, the system demonstrates a robust implementation of sequential control logic for an industrial setup.

Conclusion

The sequential control system was implemented and tested successfully. The combination of input/output controls, timers, and state transitions enabled smooth operation of the motor and visual feedback through the lamp. The modular design ensures scalability, clarity, and ease of debugging. The provided photos demonstrate the hardware in action during execution, confirming that the system behaves as expected under all tested conditions.